

menus. That said, because it's likely the user will have a keyboard, you may choose to make some of your touch optimisation work include increasing keyboard accelerators for driving the menu, thereby generally reducing the need to use the menu.

In terms of toolbars, you may find that the buttons are too small. Toolbars are necessary, but developers tend to make them as small as possible, because they are a secondary feature compared to the display of the data you're working on.

The bigger problem with toolbars is tooltips. Introduced with Microsoft Windows 95, these tips allow users to use the mouse to obtain a deeper understanding of what a button does. Toolbar icons are often cryptic, and often their meaning is established by convention rather than meaningful icon. Consider the often-used floppy icon for saving files. To most people of this generation that icon bears no resemblance to anything they use, for them it is the save icon rather than the image of a floppy. A new user then would have to look at a tooltip over this button to understand what it means rather than by looking at that icon.

The Intel team's research didn't include a test as to whether users took advantage of tooltips, but it's easy to assume that users wanting to know what a button did might well hold it down and expect more information to appear. This functionality is certainly doable, but consider that the user's finger and hand will be obscuring the screen around the button, etc. Therefore, you will need to consider where you display the tooltip. As tooltips are a standard feature of most toolbar libraries, this change of presentation may require additional work.

Traditional desktop applications often rely on pop-up forms and dialog boxes. The only consideration here is that people often want to move and resize these items. The operating system will, of course, do this for you. Moreover, the operating system should do a good job by itself of handling touch interaction on captions and borders. Therefore, verify that your application works adequately in this regard out of the box.

OTHER APPLICATIONS

So much for basic forms applications. What about those applications with more novel, specialised interfaces?

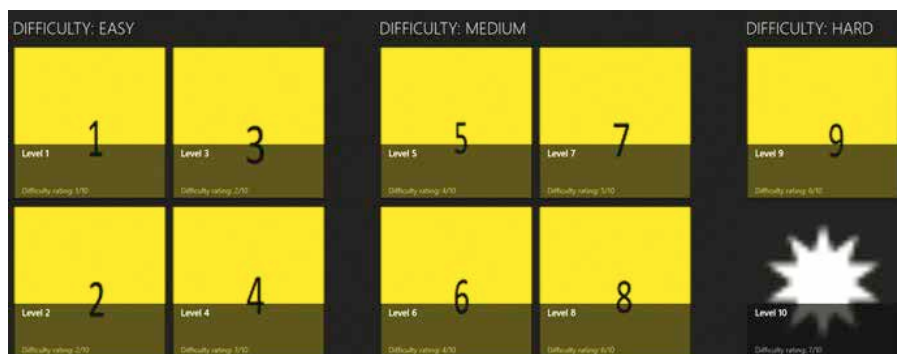
From time to time, developers build custom interfaces with the expectation that users will drag items around on their surfaces or scroll around. Consider an image editing application; you are working on a canvas, where the image can be moved around to put focus on different areas. You can add elements to the image, and resize or rotate them. In such a case you need to ensure that the handles for resizing, and rotation are large enough to be picked up via touch.

You may run into a problem where the operating system is limited in the help it can give you in interpreting touch placement. When the operating system is asked to display a form, it has an understanding of the construction of that form and therefore is able to make decisions like, "The user clicked in a blank area; I suppose he or she actually wanted to click this field." If you custom

Gestures such as pinch-to-zoom aren't something that can be replicated with a mouse though, so it is important to not have them as the exclusive means to perform an action. Otherwise you will leave behind mouse users.

STUDY: CHILDREN'S MATH GAME

In this game, the user is given the option to choose a difficulty level. In every level, the user selects operands (digits) and operators (+, -, *, /). The goal is to construct an arithmetic sequence that uses both operators and operands to create a sound mathematical equation. If the equation is correct, the tiles disappear; otherwise, nothing happens. The goal is to make all operand tiles disappear. It is possible to get stuck in a level if equations aren't constructed properly! Thus, the game requires thinking ahead.



The user simply taps on each tile to make the selection

draw the whole interface, the operating system just sees a rendering surface without deeper meaning. As such, you may need to replicate the fuzzy matching that the operating system does on your own custom controls.

ZOOMING AND SCROLLING

Operating systems typically interpret scroll intentions properly. A word of caution, though. If you use third-party control libraries in your application, you may run into problems. Although the operating system knows that one of its list box controls is a list box and can infer a scroll intention, that may not be a given for a third-party control. That said, it should generally be the case that scrolling works alright, as even third-party controls tend to build on and extend the regular operating system controls.

One area where you may find yourself working harder is zooming. "Pinch to zoom" on touch devices is natural, but there's no built-in support for it on the standard controls.

When the app is launched, the user sees:

If, for instance, the user chooses level 3, the game board XAML is loaded and play begins:

On this game board, a correct equation could be constructed as follows: $(1+2)-2 = 1$. Note that the game does not use the formal order of operations—equations are processed from left to right. So, here, the user could choose the following tiles in this order: 1, +, 2, -, 2, 1

The game does not need an equal sign; the game logic tracks the cumulative total as the user continues selecting tiles. The following picture shows the partial selection of all but the final tile. The selected tiles are shaded so they appear darker. Now, when the user selects "1", the equation is properly constructed, and the tiles disappear!

There is one problem. The game has no way to arithmetically eliminate all remaining operand tiles. Winning in the original level structure shown above is left as an exercise for the user! ■